# Restaurant Case Study Using Hadoop and PySpark

**1. Global Cuisine Popularity Analysis :**

The Problem: The Cuisines column contains comma-separated values (e.g., "French, Japanese, Desserts").
Goal: Calculate the total count of each cuisine globally and per city to identify the most popular food types.

```
>>> df = spark.read.csv("/jaya/resturant",header=True,inferSchema=True)
>>> df.show()
```

```
>>> from pyspark.sql.functions import *
>>> cuisine_df = df.withColumn("Cuisine",
...     explode(split(col("Cuisines"),","))
... )
>>> cuisine_df = cuisine_df.withColumn(
...     "Cuisine",
...     trim(col("Cuisine"))
... )
```

******* The total count of each cuisine globally *******

```
>>> cuisine_df.groupBy("Cuisine") \
... .count() \
... .orderBy(desc("count")) \
... .show()
+------------+-----+
|     Cuisine|count|
+------------+-----+
|North Indian| 3958|
|     Chinese| 2733|
|   Fast Food| 1984|
|     Mughlai|  994|
|     Italian|  763|
|      Bakery|  745|
| Continental|  736|
|        Cafe|  703|
|    Desserts|  653|
|South Indian|  635|
| Street Food|  562|
|    American|  389|
```

```
|      Mithai|  380|
|       Pizza|  379|
|      Burger|  250|
|        Thai|  234|
|       Asian|  233|
|   Beverages|  229|
|   Ice Cream|  226|
|     Mexican|  180|
+------------+-----+
```

******* The total count of each cuisine per city *******

```
>>> cuisine_df.groupBy("City","Cuisine") \
... .count() \
... .orderBy(desc("count")) \
... .show()
+---------+------------+-----+
|     City|     Cuisine|count|
+---------+------------+-----+
|New Delhi|North Indian| 2425|
|New Delhi|     Chinese| 1638|
|New Delhi|   Fast Food| 1303|
|New Delhi|     Mughlai|  628|
|    Noida|North Indian|  530|
|  Gurgaon|North Indian|  508|
|New Delhi|      Bakery|  465|
|New Delhi|South Indian|  411|
|New Delhi| Street Food|  411|
|New Delhi|    Desserts|  383|
|    Noida|     Chinese|  382|
|New Delhi|     Italian|  376|
```

```
|New Delhi| Continental|  373|
|New Delhi|        Cafe|  325|
|  Gurgaon|     Chinese|  324|
|New Delhi|      Mithai|  282|
|    Noida|   Fast Food|  265|
|  Gurgaon|   Fast Food|  220|
|New Delhi|       Pizza|  196|
|    Noida|     Mughlai|  155|
```

**2. Normalizing and Comparing Costs Across Currencies :**

The Problem: The Average Cost for two is in different local currencies (INR, Dollar, Botswana Pula, etc.).
Goal: Determine which city/country has the most expensive or affordable dining scenes on a global scale.

```
currency = [
("Botswana Pula(P)",0.075),
("Indian Rupees(Rs.)",0.012),
("Dollar($)",1.0)
]
cur_df = spark.createDataFrame(
    currency,
    ["Currency","rate"]
)
from pyspark.sql.functions import broadcast
join_df = df2.join(
    broadcast(cur_df),
    "Currency",
    "left"
)
cost_df = join_df.withColumn(
    "cost_usd",
    col("cost") * col("rate")
```

******* city the most expensive or affordable dining scenes on a global scale *******

```
cost_df.groupBy("City") \
.avg("cost_usd") \
.orderBy(desc("avg(cost_usd)")) \
.show()
```

```
+----------------+------------------+
|            City|     avg(cost_usd)|
+----------------+------------------+
|      Pasay City|             250.0|
|       Singapore|158.94736842105263|
|Mandaluyong City|             150.0|
|     Taguig City|           129.375|
|     Paynesville|             120.0|
|   Tagaytay City|              90.0|
|     Makati City|             86.25|
|Vineland Station|              70.0|
|       Princeton|              70.0|
|      Pasig City|              70.0|
|   San Juan City|             63.75|
|      Santa Rosa|            61.875|
|     Quezon City|              60.0|
|         Orlando|             40.25|
|    Ojo Caliente|              40.0|
|       Tampa Bay|             34.75|
```

******* Country the most expensive or affordable dining scenes on a global scale *******

```
cost_df.groupBy("Country Code") \
.avg("cost_usd") \
.orderBy(desc("avg(cost_usd)")) \
.show()
-----------------+------------------+
     Country Code|     avg(cost_usd)|
-----------------+------------------+
              184|158.94736842105263|
              162|120.51136363636364|
               37|             36.25|
              216|26.152073732718893|
               14|24.083333333333332|
                1|7.4808510638297765|
```

**3. Impact of Services on Customer Ratings :**

The Problem: Does having "Online Delivery" or "Table Booking" actually lead to better reviews?
Goal: Provide business insights into whether investing in delivery infrastructure correlates with higher customer satisfaction.
******* Has online delivery *******

```
df2.groupBy("Has Online delivery") \
.avg("rating") \
.show()
----------------+------------------+
 Online delivery|       avg(rating)|
----------------+------------------+
               0|              null|
            null|              null|
              No|2.4656236786469345|
             Yes|3.2486938775510255|
             3.2|              null|
----------------+------------------+
```

******* Has table booking *******

```
df2.groupBy("Has Table booking") \
.avg("rating") \
.show()
--------------+------------------+
 Table booking|       avg(rating)|
--------------+------------------+
          null|              null|
            No|  2.55956837963515|
           Yes|3.4419689119170966|
             1|              null|
             4|              null|
--------------+------------------+
```

```
df2.groupBy(
    "Has Online delivery",
    "Has Table booking"
).avg("rating") \
.show()
----------------+-----------------+------------------+
 Online delivery|Has Table booking|       avg(rating)|
----------------+-----------------+------------------+
             3.2|                4|              null|
               0|                1|              null|
             Yes|              Yes|3.5958620689655163|
            null|             null|              null|
              No|               No|2.3653483992467024|
             Yes|               No|3.1737468982630337|
              No|              Yes|3.3493775933609933|
----------------+-----------------+------------------+
```

**4. Identifying "Hidden Gems" (Weighted Rating Analysis) :**

The Problem: Some restaurants have a 5.0 rating but only 2 votes, while others have

4.5 with 2,000 votes.
Goal: Find highly-rated restaurants that have significant customer trust.

```
C = data["Aggregate rating"].mean()
m = data["Votes"].quantile(0.75)
data["Weighted_Rating"] = (
    (data["Votes"]/(data["Votes"]+m) * data["Aggregate rating"]) +
    (m/(data["Votes"]+m) * C)
)
hidden_gems = data[
    (data["Votes"] >= m) &
    (data["Aggregate rating"] >= 4.0)
]
hidden_gems = hidden_gems.sort_values(
    by="Weighted_Rating",
    ascending=False
)
hidden_gems[
    ["Restaurant Name","City","Aggregate rating","Votes","Weighted_Rating"]].head(10)
```

| | Restaurant Name | City | Aggregate rating | Votes | Weighted_Rating |
|---|---|---|---|---|---|
| **2414** | Barbeque Nation | Kolkata | 4.9 | 5966 | 4.852008 |
| **2307** | AB's - Absolute Barbecues | Hyderabad | 4.9 | 5434 | 4.847420 |
| **2495** | Mirchi And Mime | Mumbai | 4.9 | 3244 | 4.813302 |
| **3013** | Naturals Ice Cream | New Delhi | 4.9 | 2620 | 4.793637 |
| **374** | McGuire's Irish Pub & Brewery | Pensacola | 4.9 | 2238 | 4.776486 |
| **9296** | Talaga Sampireun | Tangerang | 4.9 | 2212 | 4.775115 |
| **728** | Toit | Bangalore | 4.8 | 10934 | 4.774740 |
| **3601** | Indian Accent - The Manor | New Delhi | 4.9 | 1934 | 4.758302 |
| **2409** | Barbeque Nation | Kolkata | 4.9 | 1753 | 4.744689 |
| **9279** | Talaga Sampireun | Jakarta | 4.9 | 1662 | 4.736807 |

**5. Restaurant Clustering for Marketing (PySpark MLlib) :**

The Problem: Grouping restaurants for targeted advertisements.
Goal: Categorize restaurants into segments like "Luxury Dining," "Budget Eats," and "Popular Mid-range."

```
df = data[[
    "Price range",
    "Aggregate rating",
    "Votes"
]].dropna()
scaler = StandardScaler()
X = scaler.fit_transform(df)
kmeans = KMeans(n_clusters=3, random_state=0)
df["Cluster"] = kmeans.fit_predict(X)
cluster_names = {
    0: "Mid-range",
    1: "Budget",
    2: "Luxury"
}
df["Segment"] = df["Cluster"].map(cluster_names)
print(df.head())
print(kmeans.cluster_centers_)
```

```
   Price range  Aggregate rating  Votes  Cluster    Segment
0            3               4.8    314        0  Mid-range
1            3               4.5    591        0  Mid-range
2            4               4.4    270        0  Mid-range
3            4               4.9    365        0  Mid-range
4            4               4.8    229        0  Mid-range
[[ 0.1372825   0.48985803 -0.06750571]
 [-0.62951297 -1.75666448 -0.36253503]
 [ 1.26458875  1.00197751  4.19701131]]
```

**6. Geolocation-based Food Hub Detection :**

The Problem: Identifying dense clusters of restaurants (Food Hubs) without relying on locality names.
Goal: Identify the exact geographic areas where restaurant competition is highest

```
from sklearn.cluster import KMeans
df = data[["Latitude","Longitude"]].dropna()
kmeans = KMeans(n_clusters=5, random_state=0)
df["Hub"] = kmeans.fit_predict(df)
print(df.head())
print(kmeans.cluster_centers_)
```

```
    Latitude   Longitude  Hub
0  14.565443  121.027535    0
1  14.553708  121.014101    0
2  14.581404  121.056831    0
3  14.585318  121.056475    0
4  14.584450  121.057508    0
[[ 27.76392648  77.32669172]
 [ 35.72189937 -92.97461685]
 [ -4.80249878  -2.04482399]
 [ 49.17178324   8.16757066]
 [-36.50936809 163.45694887]]
```

**7. Data Quality & Standardization Pipeline :**

The Problem: The dataset contains null values in Cuisines and potential encoding issues in city names.
Goal: Ensure the data is "Analytics Ready" for downstream business intelligence tools.

```
from pyspark.sql.functions import col, lower, trim
df_clean = df.dropna(subset=["Cuisines"]) \
    .withColumn("City", lower(trim(col("City")))) \
    .withColumn("Cuisines", lower(trim(col("Cuisines")))) \
    .dropDuplicates()
df_clean = df_clean.fillna({"Aggregate rating": 0})
```

**8. Locality-wise Price Range Distribution :**

The Problem: Analyzing the economic profile of different neighborhoods.
Goal: Generate a report showing the percentage of "Cheap" vs "Expensive" restaurants in every locality to help new entrepreneurs find market gaps.

```
>>> df2.filter(
...     col("segment").isin("Cheap","Expensive")
... ).groupBy(
...     "Locality",
...     "segment"
... ).count().show()
+--------------------+---------+-----+
|            Locality|  segment|count|
+--------------------+---------+-----+
|         Cedar Falls|Expensive|    2|
|  CITY WALK, Al Safa|Expensive|    1|
|Koramangala 7th B...|Expensive|    1|
|         Gopalapuram|    Cheap|    1|
|Anil Plaza, Chris...|Expensive|    2|
|DLF Emporio Mall,...|Expensive|    2|
|Hotel Classic Dip...|Expensive|    1|
|The Royal Retreat...|Expensive|    1|
|         Green Point|Expensive|    2|
|           Inverloch|    Cheap|    1|
|             Al Khan|Expensive|    1|
|       Rajguru Nagar|Expensive|    1|
```

**9. Price Range Prediction (Classification) :**

The Problem: Can we predict the Price range (1 to 4) of a new restaurant based on its location and cuisine?
Goal: Build a model that predicts the category of a restaurant based on its basic profile.

```python
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
df = data[[
    "Aggregate rating",
    "Votes",
    "Latitude",
    "Longitude",
    "Price range"
]].dropna()
X = df[[
    "Aggregate rating",
    "Votes",
    "Latitude",
    "Longitude"
]]
y = df["Price range"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=0
)
model = RandomForestClassifier()
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

```
0.5766614338042909
```

**10. Top-N Recommendation Engine (Content-Based) :**

The Problem: Recommending similar restaurants to a user.
Goal: Given a restaurant the user likes, recommend 5 other similar restaurants in the same city.

```python
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
df = data[[
    "Restaurant Name",
    "City",
    "Cuisines"
]].dropna()
cv = CountVectorizer()
X = cv.fit_transform(df["Cuisines"])
similarity = cosine_similarity(X)
def recommend(name):
    idx = df[df["Restaurant Name"] == name].index[0]
    city = df.loc[idx, "City"]
    scores = list(enumerate(similarity[idx]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)
    result = []
    for i in scores[1:]:
        if df.iloc[i[0]]["City"] == city:
```

```python
        if df.iloc[i[0]]["City"] == city:
            result.append(df.iloc[i[0]]["Restaurant Name"])
        if len(result) == 5:
            break
    return result
print(recommend(df.iloc[0]["Restaurant Name"]))
```

```
['Izakaya Kikufuji']
```

**11. Optimized Currency Conversion (Broadcast Join) :**

The Problem: Joining a massive restaurant dataset with a tiny 15-row currency exchange table triggers a standard shuffle join, moving millions of rows across the network unnecessarily
Goal: Implement a Broadcast Join to eliminate network shuffle and speed up the cost
normalization process across different global currencies.

```
currency_data = [
("Botswana Pula(P)",0.075),
("Indian Rupees(Rs.)",0.012),
("Dollar($)",1.0)
]
cur_df = spark.createDataFrame(
    currency_data,
    ["Currency","rate"]
)
cost_df = df.join(
    broadcast(cur_df),
    "Currency",
    "left"
).withColumn(
    "cost_usd",
    col("Average Cost for two") * col("rate")
)
cost_df.select(
    "Restaurant Name",
    "City",
    "Currency",
```

```
>>> cost_df.select(
...     "Restaurant Name",
...     "City",
...     "Currency",
...     "Average Cost for two",
...     "cost_usd"
... ).show()
+--------------------+----------------+----------------+--------------------+--------+
|     Restaurant Name|            City|        Currency|Average Cost for two|cost_usd|
+--------------------+----------------+----------------+--------------------+--------+
|    Le Petit Souffle|     Makati City|Botswana Pula(P)|                1100|    82.5|
|    Izakaya Kikufuji|     Makati City|Botswana Pula(P)|                1200|    90.0|
|Heat - Edsa Shang...|Mandaluyong City|Botswana Pula(P)|                4000|   300.0|
|                Ooma|Mandaluyong City|Botswana Pula(P)|                1500|   112.5|
|         Sambo Kojin|Mandaluyong City|Botswana Pula(P)|                1500|   112.5|
|        Din Tai Fung|Mandaluyong City|Botswana Pula(P)|                1000|    75.0|
|          Buffet 101|      Pasay City|Botswana Pula(P)|                2000|   150.0|
|             Vikings|      Pasay City|Botswana Pula(P)|                2000|   150.0|
|Spiral - Sofitel ...|      Pasay City|Botswana Pula(P)|                6000|   450.0|
```

```
|                Ooma|Mandaluyong City|Botswana Pula(P)|                1500|   1
|         Sambo Kojin|Mandaluyong City|Botswana Pula(P)|                1500|   1
|        Din Tai Fung|Mandaluyong City|Botswana Pula(P)|                1000|    
|          Buffet 101|      Pasay City|Botswana Pula(P)|                2000|   1
|             Vikings|      Pasay City|Botswana Pula(P)|                2000|   1
|Spiral - Sofitel ...|      Pasay City|Botswana Pula(P)|                6000|   4
|            Locavore|      Pasig City|Botswana Pula(P)|                1100|    
|    Silantro Fil-Mex|      Pasig City|Botswana Pula(P)|                 800|    
|Mad Mark's Creame...|      Pasig City|Botswana Pula(P)|                 900|    
|    Silantro Fil-Mex|     Quezon City|Botswana Pula(P)|                 800|    
|          Guevarra's|   San Juan City|Botswana Pula(P)|                1000|    
|Sodam Korean Rest...|   San Juan City|Botswana Pula(P)|                 700|    
|       Cafe Arabelle|      Santa Rosa|Botswana Pula(P)|                 800|    
|Nonna's Pasta & P...|      Santa Rosa|Botswana Pula(P)|                 850|   6
|          Balay Dako|   Tagaytay City|Botswana Pula(P)|                1200|    
|Hobing Korean Des...|     Taguig City|Botswana Pula(P)|                 600|    
|Wildflour Cafe + ...|     Taguig City|Botswana Pula(P)|                1500|   1
+--------------------+----------------+----------------+--------------------+-----
```

**12. Efficient Data Pipelines (Caching & Storage) :**

The Problem: The "Cuisine Popularity" analysis and "Price Range Prediction" both use the same cleaned dataset. Without optimization, Spark re-reads the raw CSV from HDFS and re-calculates the split() and explode() transformations for every single query.
Goal: Use .cache() to persist intermediate cleaned data in memory and save the final "Analytics Ready" dataset in Parquet format to leverage columnar storage and compression.

```
>>> from pyspark.sql import SparkSession
>>> from pyspark.sql.functions import col, lower, trim
>>> df_clean = df.dropna(subset=["Cuisines"]) \
..             .withColumn("City", lower(trim(col("City")))) \
..             .withColumn("Cuisines", lower(trim(col("Cuisines")))) \
..             .dropDuplicates()
>>> df_clean.cache()
DataFrame[Restaurant ID: string, Restaurant Name: string, Country Code: string, City: string, Address: string, Locality: str
cality Verbose: string, Longitude: string, Latitude: string, Cuisines: string, Average Cost for two: string, Currency: strin
Table booking: string, Has Online delivery: string, Is delivering now: string, Switch to order menu: string, Price range: st
ggregate rating: string, Rating color: string, Rating text: string, Votes: int]
>>> df_clean.groupBy("Cuisines").count().show()
--------------------+-----+
            Cuisines|count|
--------------------+-----+
north indian, chi...|  511|
north indian, mug...|  334|
cafe, kiwi, ice c...|    1|
cafe, north india...|    1|
bbq, caribbean, s...|    1|
```

```
>>> df_clean.groupBy("Price range").count().show()
+-----------+-----+
|Price range|count|
+-----------+-----+
|          3| 1406|
|         30|    1|
|          0|    2|
|          1| 4437|
|     Yellow|    2|
|          4|  585|
|          2| 3109|
+-----------+-----+

>>> df_clean.write.mode("overwrite").parquet("cleaned_restaurant_data.parc
```

**13. Service Impact Comparative Visualization :**

The Problem: Simply listing the average rating for restaurants with "Online Delivery" vs. "Table Booking" doesn't show the full story of customer satisfaction or the volume of feedback.
Goal: Create a Side-by-Side Box Plot to compare rating distributions for each service, using Color Overlays to represent different price ranges and Point Density (Jitter) to show the total number of votes.

```
>>> df_vis = df.select(
...     "Has Online delivery",
...     "Has Table booking",
...     "Aggregate rating",
...     "Votes",
...     "Price range"
... )
>>> from pyspark.sql.functions import col
>>> df_vis = df_vis.withColumn(
...     "rating",
...     col("Aggregate rating").cast("double")
... ).withColumn(
...     "votes",
...     col("Votes").cast("int")
... )
>>> df_vis.groupBy(
...     "Has Online delivery",
...     "Has Table booking",
```

```
>>> df_vis.groupBy(
...     "Has Online delivery",
...     "Has Table booking",
...     "Price range"
... ).avg("rating","votes").show()
+------------------+----------------+-----------+------------------+---------------
|Has Online delivery|Has Table booking|Price range|       avg(rating)|        avg(vot
+------------------+----------------+-----------+------------------+---------------
|                 No|              No|          4| 4.015100671140939| 397.7953020134
|               null|            null|       null|              null|              n
|                  0|               1|          0|              null|              n
|                3.2|               4|         30|              null|              n
|                Yes|             Yes|          1|               3.7|              6
|                Yes|              No|          3| 3.622857142857143|498.32142857142
|                 No|             Yes|          3|   3.4112600536193| 346.9490616621
|                 No|              No|          1|1.8102618920363396|37.703367183324
|                 No|              No|          2|2.7392857142857134|132.69086651053
|                Yes|             Yes|          3|3.6302583025830266|494.23247232472
|                 No|              No|          3| 3.882692307692307| 467.6955128205
|                 No|              No|     Yellow|              null|              n
|                Yes|             Yes|          2|3.4268292682926833| 317.6991869918
```

```
>>> df_vis.groupBy(
...     "Has Table booking",
...     "Price range"
... ).avg("rating").show()
+----------------+-----------+------------------+
|Has Table booking|Price range|       avg(rating)|
+----------------+-----------+------------------+
|               4|         30|              null|
|             Yes|          3|3.5034161490683204|
|            null|       null|              null|
|             Yes|          2|3.1020920502092033|
|              No|          1| 1.999954975236381|
|              No|          3| 3.835078534031416|
|              No|          2|2.9281184668989573|
|              No|          0|              null|
|             Yes|          4| 3.593065693430658|
|             Yes|          1|               3.7|
|               1|          0|              null|
|              No|          4| 4.018006430868167|
|              No|     Yellow|              null|
```